ELEKTRİKLİ ARAÇ ARAÇ İÇİ HABERLEŞME

ÖN KAPAK

İÇİNDEKİLER

1. TAKIM BİLGİSİ
2. KAPAK/GİRİŞ ve DİNAMİK BELLEK YÖNETİMİ ANALİZİ
3. KUYRUK FONKSİYONLARININ ANALİZİ
4. TEORİK MİMARİ KARARLAR
5. TAKIM ÇALIŞMASI VE MİMARİ KARARLAR

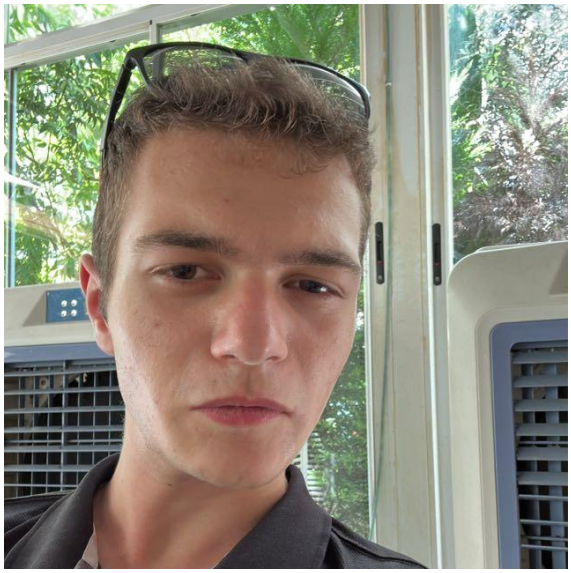
TAKIM BİLGİSİ

OKAN DOĞAN

- main.c düzenleme
- github kurulumları ve repo açıcısı

DEMİRHAN HACIVELİOĞLU

- can_queue.h ve can_queue.c düzenleme
- yorum satırları düzenleyicisi
- ekibin gözbebeği, geleceği ve genç yeteneği



1. Projenin Amacı ve Kapsamı Bu projenin temel amacı, modern elektromobillerde sensörler (hız, batarya, motor sıcaklığı vb.) ile Elektronik Kontrol Ünitesi (ECU) arasındaki veri akışını sağlayan CAN Bus (Controller Area Network) iletişim hattı için bir simülasyon geliştirmektir. Gerçek dünya senaryolarında birden fazla sensörün aynı anda veri göndermesi durumunda veri kaybını önlemek kritik bir gereksinimdir. Bu doğrultuda, sensörlerden gelen mesajların kaybolmadan sırayla işlenebilmesi için İlk Giren İlk Çıkar (FIFO - First In, First Out) mantığına dayalı bir mesaj kuyruğu (Queue) veri yapısı tasarlanmıştır. Geliştirilen bu yapı, Linked List kullanılmadan, çalışma zamanında (runtime) kapasitesi belirlenebilen dinamik bir dizi işaretçisi (pointer) ile yönetilmektedir.

2. Kod Mimarisi ve Dinamik Bellek Yönetimi (can_queue.c İncelemesi) Projenin temel iş mantığı can_queue.c dosyası içerisinde modüler bir yapıda kurgulanmıştır. Sabit boyutlu veri yapılarının getirdiği bellek israfı veya yetersiz alan problemlerini aşmak adına Dinamik Bellek Yönetimi (Dynamic Memory Allocation) tercih edilmiştir.

2.1. Kuyruğun Başlatılması ve Bellek Tahsisi (initQueue) Sistem başlatıldığında ilk olarak initQueue fonksiyonu çağrılmaktadır. Bu fonksiyon, kullanıcının belirlediği kapasite değerini referans alarak bellekte tam olarak ihtiyaç duyulan alanı ayırır.

- Fonksiyon, MesajKuyrugu yapısı (struct) içindeki kapasite değişkenine kullanıcının girdiği boyutu atar.
- Kuyruğun kalbini oluşturan mesajlar dizisi için malloc (Memory Allocation) fonksiyonu kullanılır. Bellekte kapasite * sizeof(CAN_Mesaj) formülü ile her bir sensör mesajının bellekte kapladığı alan hesaplanarak toplam kapasite kadar sürekli (contiguous) bir bellek bloğu tahsis edilir.
- Bu işlemden sonra kuyruğun yönlendirme değişkenleri (pointers) sıfırlanır: İşlenecek ilk mesajın indeksini tutan front değişkeni 0'a, eklenecek yeni mesajın indeksini tutan rear değişkeni -1'e ve anlık mesaj sayısını takip eden count değişkeni 0'a eşitlenerek sistem veri kabulüne hazır hale getirilir.

```c
// Kuyruğu başlatma ve dinamik bellek ayırma
void initQueue(MesajKuyrugu *q, int kapasite) {
    q->kapasite = kapasite;
    // Dinamik dizi (pointer) ile bellekte alan tahsisi
    q->mesajlar = (CAN_Mesaj*)malloc(q->kapasite * sizeof(CAN_Mesaj));
    q->front = 0;
    q->rear = -1;
    q->count = 0;
}
```

2.2. Belleğin İşletim Sistemine İadesi (freeQueue) Gömülü sistem programlamada en önemli hususlardan biri bellek sızıntılarını (memory leak) önlemektir. İşletim

sistemi veya mikrodenetleyici kapanana kadar arka planda çalışan CAN Bus sistemlerinin belleği verimli kullanması zorunludur.

- Program sonlandırılmadan hemen önce freeQueue fonksiyonu devreye girer.
- malloc ile tahsis edilen ve q->mesajlar işaretçisinin gösterdiği bellek bloğu free() fonksiyonu ile RAM'e iade edilir. Bu sayede bellek sızıntısı engellenerek sistemin kilitlenmesinin önüne geçilir ve güvenli bir kapanış sağlanır.

```
// 4. Sistemi Kapat (Memory Free) [cite: 62]
void freeQueue(MesajKuyrugu *q) {
    // Dinamik olarak ayrılan belleği RAM'e iade etme
    free(q->mesajlar);
}
```

3. Kuyruk İşlemleri ve Veri Akış Mantığı (can_queue.c İncelemesi - Devam)
Sistemin dinamik bellekte yerini almasının ardından, sensör mesajlarının sıraya alınması ve işlenmesi süreci başlar. Bu süreçte veri yapısının dairesel dizi (Circular Array) olarak yönetilmesi, bellek verimliliği açısından kilit rol oynamaktadır.

3.1. Sensör Mesajının Kuyruğa Eklenmesi (enqueue) Sensörden bir veri geldiğinde, bu verinin kuyruğun sonuna güvenli bir şekilde eklenmesi gerekmektedir. enqueue fonksiyonu bu işlemi şu adımlarla gerçekleştirir:

1. **Taşma Kontrolü (Edge Case - Queue Overflow):** Ekleme işleminden önce q->count == q->kapasite kontrolü yapılır. Eğer kuyruk doluysa, "[HATA]" mesajı ile kullanıcı uyarılır ve veri kaybını önlemek adına yeni mesaj eklenmeden fonksiyondan çıkılır. Bu kontrol, gömülü sistemlerin kilitlenmesini önleyen kritik bir emniyet mekanizmasıdır.
2. **Dairesel İndeks Hesaplaması:** Kuyrukta yer varsa, yeni mesajın ekleneceği rear indeksi güncellenir. Burada standart dizi mantığı yerine **modüler aritmetik** (q->rear = (q->rear + 1) % q->kapasite) kullanılır. Bu sayede, dizi sonuna ulaşıldığında rear indeksi tekrar başa (0) dönerek bellekteki boşlukların tekrar kullanılmasını sağlar.
3. **Veri Girişi:** Kullanıcıdan scanf ile alınan Sensör Adı, Mesaj ID'si ve Veri değeri, doğrudan q->mesajlar[q->rear] konumundaki CAN_Mesaj struct'ının ilgili alanlarına yazılır.
4. **Sayaç Güncelleme:** Mesaj başarıyla eklendikten sonra q->count değeri bir artırılır ve güncel mesaj sayısı "[BİLGİ]" etiketiyle kullanıcıya sunulur.

3.2. Bekleyen İlk Mesajın İşlenmesi (dequeue) Ana bilgisayar (ECU) boşta olduğunda, kuyrukta bekleyen en eski mesajı (İlk Giren) alıp işler. dequeue fonksiyonu bu süreci yönetir:

1. **Boş Kuyruk Kontrolü (Edge Case - Queue Underflow):** Eğer q->count == 0 ise, işlenecek mesaj olmadığına dair "[BİLGİ]" mesajı verilir ve işlem sonlandırılır.
2. **Mesajın Alınması:** front indeksinin gösterdiği yerdeki CAN_Mesaj, islenen adında geçici bir struct'a kopyalanır.
3. **Verinin İşlenmesi (Yazdırılması):** Mesajın içeriği (Gönderen, ID ve Veri), PDF'teki çıktı formatına uygun olarak ekrana yazdırılır. Bu aşama, gerçek bir sistemde verinin ECU tarafından yorumlanıp araca komut verilmesini simüle eder.
4. **Front İndeksinin İlerletilmesi:** Mesaj "çıkarıldıktan" sonra, sıradaki mesajın işlenmesi için front indeksi dairesel mantıkla (q->front = (q->front + 1) % q->kapasite) bir sonraki konuma kaydırılır.
5. **Sayaç Güncelleme:** Kuyruktaki anlık mesaj sayısını tutan q->count değeri bir azaltılır.

```c
// 2. Bekleyen İlk Mesajı İşle (Dequeue)
void dequeue(MesajKuyrugu *q) {
    // Sınır Durumu: Boş Kuyruk (Queue Underflow)
    if (q->count == 0) {
        printf("[BİLGİ]: İşlenecek yeni mesaj bulunmuyor.\n");
        return;
    }
```

3.3. Bekleyen Mesajların Listelenmesi (printQueue) Bu fonksiyon, kuyruğun anlık durumunu görselleştirmek için kullanılır. PDF'teki yönergelere uygun olarak geliştirilen algoritma şu şekildedir:

- Fonksiyon, for döngüsü ile kuyrukta anlık bulunan q->count kadar eleman üzerinde döner.
- **Kritik Fark:** Döngü 0'dan başlamaz. Bunun yerine, işlenecek ilk eleman olan q->front'tan başlar ve dairesel dizeyi doğru sırayla dolaşabilmek için her adımda gerçek indeks (q->front + i) % q->kapasite formülü ile hesaplanır. Bu sayede, kuyruğa ekleme sırasına göre doğru bir LIFO listesi oluşturulur.

```c
// 3. Bekleyen Mesajları Göster (Print Queue)
void printQueue(MesajKuyrugu *q) {
    // Kuyruktaki anlık count sayısını yazdırma
    printf("\n[BEKLEYEN MESAJLAR Toplam: %d]\n", q->count);
    if (q->count == 0) {
        return;
    }

    // İşlenecek ilk elemandan (front) başlayarak en son eklenen elemana (rear) kadar dolaşan döngü
    for (int i = 0; i < q->count; i++) {
        int index = (q->front + i) % q->kapasite;
        printf("%d. %s (ID: %d) Veri: %.2f\n", i + 1, q->mesajlar[index].gonderen, q->mesajlar[index].mesaj_id, q->mesajlar[index].veri);
    }
}
```

4. Mimari Kararların Analizi ve Teorik Soruların Yanıtları

Bu bölümde, projenin tasarım aşamasında alınan kritik teknik kararların gerekçeleri ve bilgisayar bilimleri açısından analizleri sunulmuştur.

4.1. Sabit Boyutlu Dizi Yerine Dinamik Bellek (malloc) Tercihinin Nedeni ve Avantajları PDF'te yöneltilen "Neden sabit boyutlu bir dizi yerine malloc ile dinamik bellek ayırmayı tercih ettik?" sorusuna cevaben, gömülü sistemler ve araç teknolojileri açısından temel gerekçeler şunlardır:

- **Esneklik ve Çalışma Zamanı (Runtime) Yapılandırması:** Sabit boyutlu bir dizi (örn: CAN_Mesaj kuyruk[100];), kod derlendiği anda (compile-time) bellekte yer kaplar. Bu durum, aracın farklı çalışma koşullarına uyum sağlamasını engeller. malloc kullanımı sayesinde, araç sahibi veya bir teknik servis, simülasyon başlarken aracın mevcut sensör yoğunluğuna veya ECU'nun işlem gücüne göre kuyruk kapasitesini anlık olarak belirleyebilir. Örneğin, yoğun şehir içi trafikte daha büyük bir kuyruk gerekirken, otoban sürüşünde daha küçük bir kuyruk yeterli olabilir.
- **Bellek Verimliliği (RAM Yönetimi):** Araç içi kontrol üniteleri (ECU), sınırlı RAM kaynaklarına sahiptir. Sabit boyutlu bir dizi, kuyruk boş olsa bile ayrılan tüm belleği işgal etmeye devam eder. malloc ise sadece ihtiyaç duyulan kadar bellek alanı ayırır. Eğer kuyruk kapasitesi 5 olarak belirlenmişse, sadece 5 mesajlık yer kaplanır. Bu, RAM israfını önler.
- **Sisteme Sağladığı Avantaj:** Bu yaklaşım, araç sistemine **ölçeklenebilirlik** (scalability) ve **kaynak optimizasyonu** sağlar. Aynı yazılım, farklı donanım özelliklerine sahip araçlarda (farklı sayıda sensör içeren modellerde) kod değişikliği gerektirmeden çalıştırılabilir. Belleğin verimli kullanılması, ECU'nun diğer kritik görevleri (örn: motor kontrolü, frenleme) için RAM ayırabilmesini garantiler ve sistemin kilitlenme riskini minimize eder.

4.2. Araç İçi Haberleşmede (CAN Bus) Neden LIFO (Stack) Yerine FIFO (Queue) Tercih Edildi? PDF'te yöneltilen "araç içi haberleşmede (CAN Bus) mesajların işlenmesi için neden LIFO (Stack) yerine... FIFO (Queue - Kuyruk) veri yapısını tercih ettik?" sorusuna cevaben, kritik zamanlı sistemlerin mantığı şu şekildedir:

- **Veri Yapısı Tanımları:** LIFO (Last In, First Out), son giren verinin ilk çıktığı yapıdır (örn: geri alma işlemi). FIFO (First In, First Out) ise ilk giren verinin ilk çıktığı, adil bir bekleme sırasıdır.
- **Gerçek Zamanlı Veri Akışı ve Olay Sıralaması:** Bir elektromobilde sensör verileri, aracın anlık durumunu kronolojik bir sıra ile yansıtır. Örneğin; tekerlek hız sensöründen gelen veri (1. mesaj), ardından batarya sıcaklık verisi (2. mesaj) gelmişse, ECU'nun bu verileri aynı kronolojik sıra ile işlemesi kritik önem taşır.

- **LIFO'nun Yarattığı Risk:** Eğer LIFO (Stack) yapısı kullanılsaydı, ECU meşgulken gelen en yeni mesaj (örn: bir hata kodu) hemen işlenirken, en eski ve belki de en kritik mesajlar (örn: fren pedalına basıldı bilgisi) stack'in en altında beklemek zorunda kalırdı. Bu durum, kritik verilerin işlenmesinde **kabul edilemez gecikmelere** (latency) ve mantıksal hatalara yol açar.
- **FIFO'nun Avantajı:** FIFO (Kuyruk) yapısı, mesajların **geliş sırasına göre adil ve deterministik** bir şekilde işlenmesini garantiler. "İlk gelen mesaj, ilk işlenir" prensibi, araç sisteminin olayları doğru bir zaman çizelgesiyle yorumlamasını sağlar. Bu, sürüş güvenliği ve sistem kararlılığı için vazgeçilmez bir gereksinimdir

4.3. Takım Arkadaşının Kodunu İnceleme (Code Review) Süreci ve Sonuç
PDF'te yöneltilen "Takım arkadaşınızın kodunu incelerken (Code Review) nelere dikkat ettiniz?" sorusuna cevaben, Pair Programming ve Pull Request (PR) aşamalarında gerçekleştirilen denetim süreçleri şunlardır:

- **Teknik Gereksinimlerin Kontrolü:** Kod incelemesi sırasında öncelikle PDF'te belirtilen temel teknik kısıtlamaların doğruluğu teyit edilmiştir. Örneğin; "Linked List KULLANILMAYACAKTIR" kuralına uyulup uyulmadığı, kuyruğun dinamik bir dizi işaretçisi (CAN_Mesaj* mesajlar) ile yönetilip yönetilmediği kontrol edilmiştir.
- **Struct Yapıları ve İsimlendirmeler:** can_queue.h dosyasındaki struct tanımlarının (CAN_Mesaj ve MesajKuyrugu) PDF'teki örnek taslak ile birebir örtüştüğü, gerekli tüm değişkenlerin (mesaj_id, veri, gonderen, kapasite, front, rear, count) doğru veri tipleriyle tanımlandığı doğrulanmıştır .
- **Hata Yönetimi ve Sınır Durumlar (Edge Cases):** Code Review sürecinin en kritik parçalarından biri, hata yönetimi mekanizmalarının test edilmesidir. enqueue fonksiyonunda "Queue Overflow" (Taşma) ve dequeue/printQueue fonksiyonlarında "Queue Underflow" (Boş Kuyruk) durumlarının doğru if kontrolleriyle ele alındığı ve kullanıcıya uygun hata mesajlarının gösterildiği teyit edilmiştir.
- **Bellek Yönetimi ve Sızıntı Kontrolü:** Projenin ana odak noktalarından biri olan bellek yönetimi titizlikle incelenmiştir. initQueue fonksiyonunda malloc ile belleğin doğru tahsis edildiği ve main.c dosyasının sonunda freeQueue fonksiyonu çağrılarak belleğin RAM'e güvenli bir şekilde iade edildiği (Memory Leak önlendiği) doğrulanmıştır.
- **Kod Kalitesi ve Okunabilirlik:** Kodun modüler yapısı, fonksiyonların tek bir sorumluluğa (SRP) sahip olması, değişken isimlendirmelerinin açıklayıcı olması ve gerekli yerlerde yorum satırlarının kullanılması gibi temiz kod (clean code) prensipleri denetlenmiştir. Bu süreç, projenin sürdürülebilirliğini ve hata ayıklama kolaylığını artırmıştır.

5. Genel Sonuç ve Değerlendirme

Bu proje kapsamında, modern elektromobillerde kritik önem taşıyan CAN Bus iletişim hattı için gerçekçi bir mesaj kuyruğu simülasyonu başarıyla geliştirilmiştir. Proje süresince şu temel kazanımlar elde edilmiştir:

1. **Dinamik Bellek Yönetimi Yetkinliği:** C dilinde malloc ve free fonksiyonları kullanılarak çalışma zamanında (runtime) bellek tahsisi ve iadesi işlemleri pratik edilmiş, sınırlı kaynaklara sahip gömülü sistemlerde RAM'in nasıl verimli yönetileceği öğrenilmiştir.
2. **Veri Yapıları Uygulaması:** FIFO (İlk Giren İlk Çıkar) prensibine dayalı Dairesel Kuyruk (Circular Queue) yapısı, sabit dizi yerine dinamik işaretçiler

kullanılarak sıfırdan implemente edilmiştir. Modüler aritmetik kullanılarak dairesel dizinin nasıl yönetileceği ve "Overflow"/"Underflow" gibi sınır durumlarının nasıl kontrol edileceği deneyimlenmiştir.

3. **Takım Çalışması ve Git Workflow:** Pair Programming ve Pull Request (PR) iş akışı sayesinde, endüstri standardı takım çalışması pratikleri uygulanmıştır. Kod incelemesi (Code Review) kültürü sayesinde, kod kalitesinin artırılması ve hataların erken tespit edilmesi süreçlerinde yetkinlik kazanılmıştır.

Geliştirilen bu simülasyon, araç içi sensör verilerinin kaybolmadan, kronolojik bir sıra ile ve adil bir şekilde işlenmesini garanti ederek, sürüş güvenliği ve sistem kararlılığı için vazgeçilmez bir altyapı sunmaktadır.